

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

SHELL SCRIPT

Autor (Samuel Cobo)

Estudiantes del curso [ITIZ 2100] – [SISTEMAS OPERATIVOS], Universidad de Las Américas, Quito-Ecuador

I. RESUMEN

En este trabajo se instaló y configuró Ubuntu en WSL y se preparó el entorno de trabajo creando la carpeta CodigosWSL. Luego se usó Visual Studio Code para editar los archivos .sh y la terminal para ejecutarlos y comprobar su funcionamiento con comandos como `bash ./EjercicioX_Cobo.sh`.

Después se desarrollaron scripts básicos y con variables/parámetros: un programa que pide el nombre y saluda, otro que muestra la fecha y hora, y varios que trabajan con argumentos, incluyendo validaciones para evitar errores cuando no se ingresan parámetros. También se implementaron expresiones matemáticas para calcular el área de un rectángulo y convertir dólares a centavos usando operaciones aritméticas en Bash.

Finalmente, se practicaron estructuras de control: un `while` que suma números hasta que el usuario ingrese 0, un `for` que imprime la tabla de multiplicar del 1 al 10, y un `case` que muestra el día de la semana según un número del 1 al 7. Para cerrar, se creó un juego de adivinar un número (1–10) con 3 intentos, validación de entrada y opción de continuar o salir (S/N), demostrando el uso combinado de condicionales, ciclos y control de flujo.

II. OBJETIVOS

Objetivo general:

Desarrollar y ejecutar scripts en Bash dentro de un entorno WSL (Ubuntu), aplicando conceptos básicos de programación en shell (entrada/salida, variables, parámetros, operaciones y estructuras de control) para resolver ejercicios prácticos.

Objetivos específicos:

1. Configurar el entorno de trabajo en WSL y utilizar VS Code/terminal para crear, editar y ejecutar archivos .sh.
2. Implementar programas en Bash usando variables, parámetros y expresiones aritméticas, incluyendo validaciones para el ingreso correcto de datos.
3. Aplicar estructuras de control (`for`, `while`, `case`) para resolver problemas y construir un juego de adivinanza con intentos limitados y opción de continuar o salir.

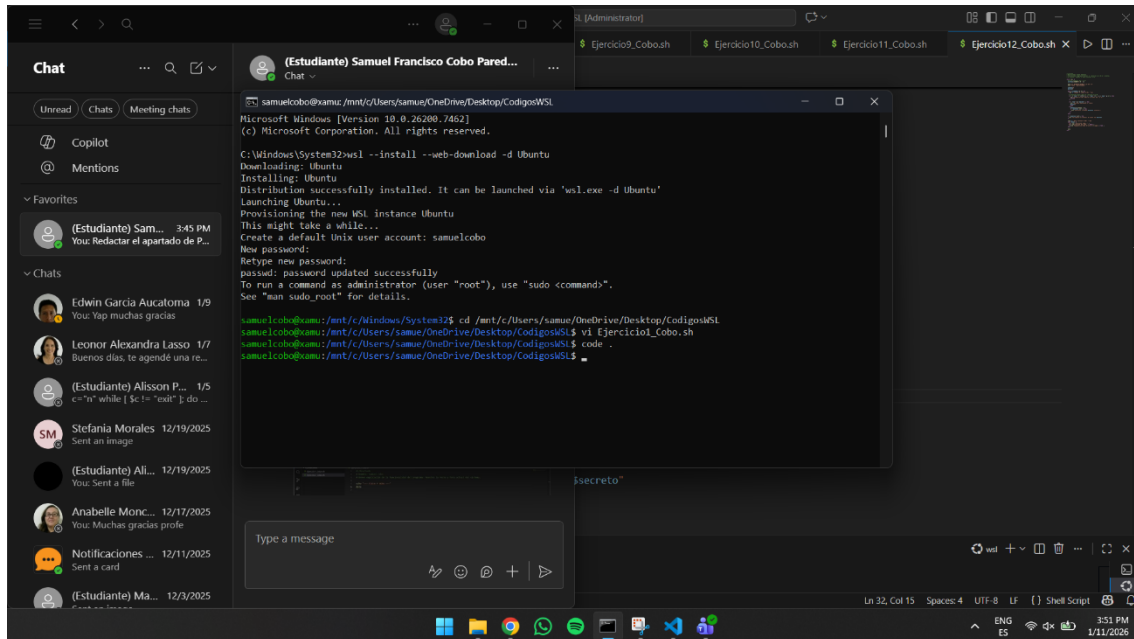
III. MATERIALES Y EQUIPOS COMPLEMENTARIOS

- Equipo de cómputo (Personal o Laboratorio)
- Simulador de terminal de Linux (Opcional)

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

IV. PROCEDIMIENTO O DESARROLLO

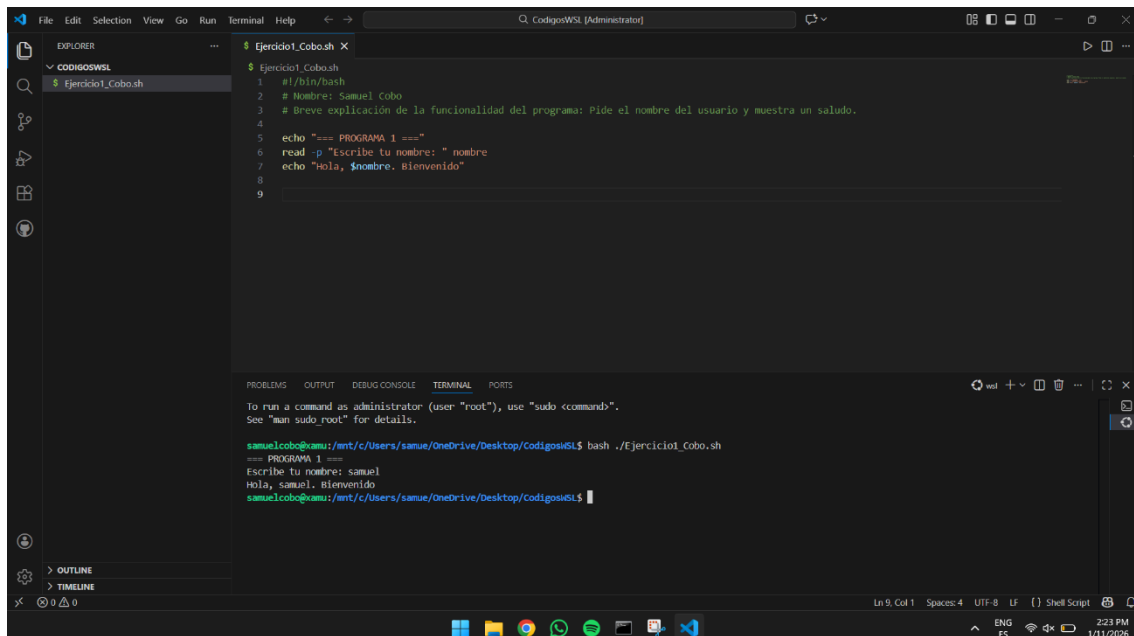
EJERCICIOS EDITOR VI EN WSL:



Paso #1

Descripción: Se instala Ubuntu en WSL desde la terminal de Windows, se crea el usuario de Linux y luego se navega a la carpeta CodigosWSL para abrir/editar el archivo Ejercicio1_Cobo.sh (con vi) y abrir el proyecto en Visual Studio Code (con code .).

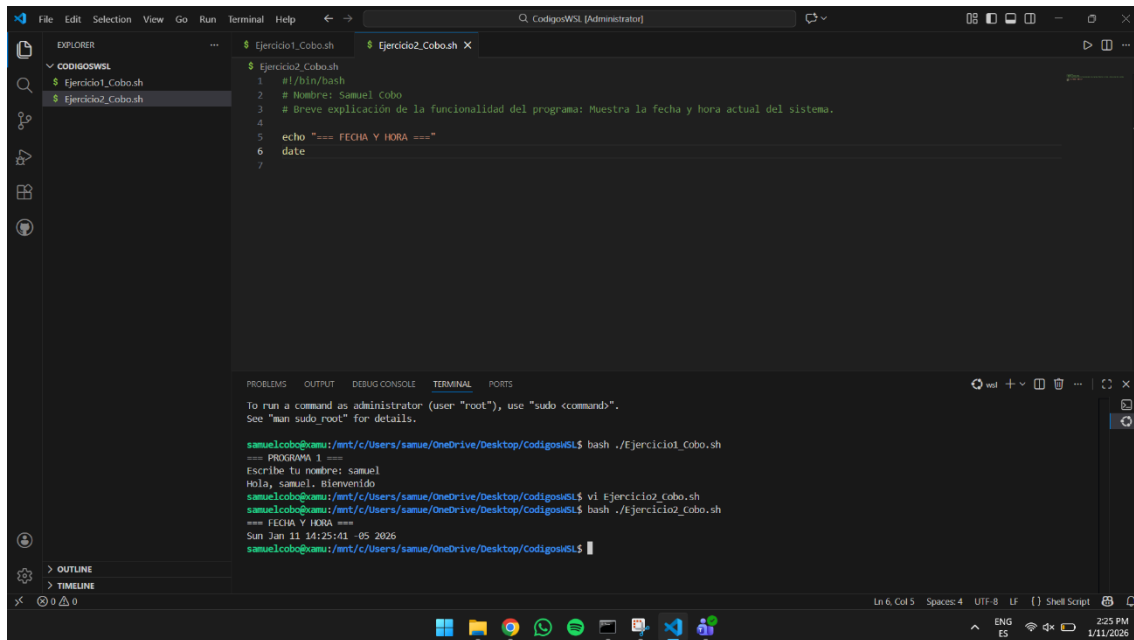
PRIMEROS PROGRAMA:



Paso #2

Descripción: Se escribe el script Ejercicio1_Cobo.sh en VS Code (con `#!/bin/bash`, comentarios y comandos `read/echo`) y luego se ejecuta desde la terminal con `bash ./Ejercicio1_Cobo.sh` para probarlo ingresando el nombre y mostrando el saludo en pantalla.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying a file named `Ejercicio2_Cobo.sh`. The main editor window shows the script content:

```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Muestra la fecha y hora actual del sistema.
4
5 echo "=== FECHA Y HORA ==="
6 date
7
```

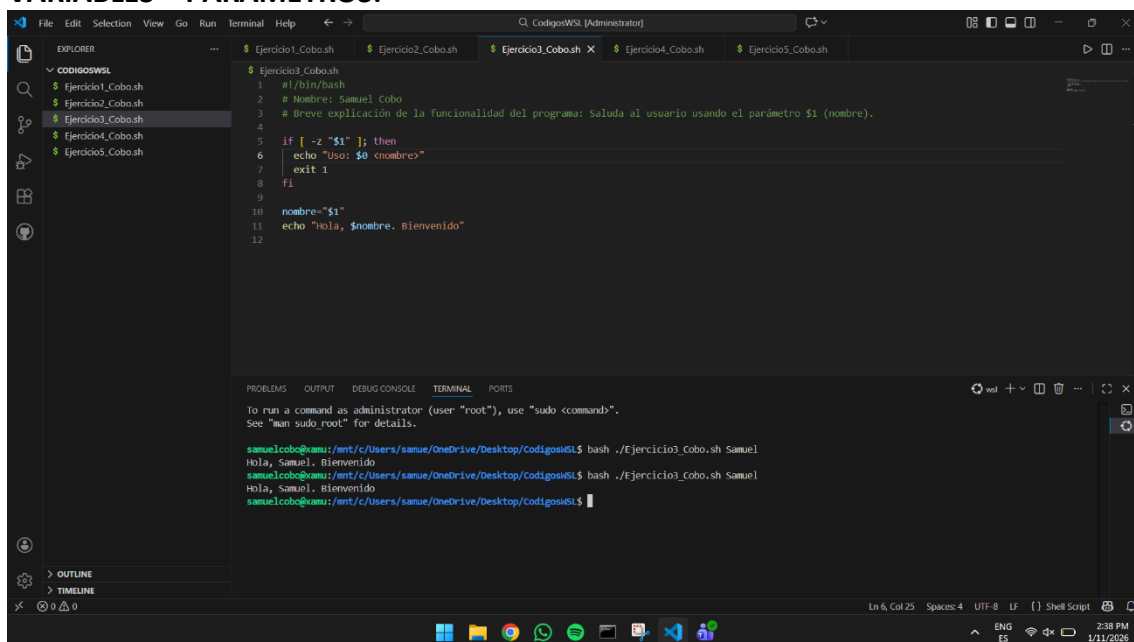
The TERMINAL panel at the bottom shows the execution of the script:

```
== PROGRAMA 1 ==
Escribe tu nombre: samuel
Hola, samuel. Bienvenido
samuelcobo@xamui:/mnt/c/Users/samue/OneDrive/Desktop/codigosWSL$ vi Ejercicio2_Cobo.sh
== FECHA Y HORA ==
Sun Jan 11 14:25:41 -05 2026
samuelcobo@xamui:/mnt/c/Users/samue/OneDrive/Desktop/codigosWSL$
```

Paso #3

Descripción: Se crea el archivo `Ejercicio2_Cobo.sh`, se escribe un script básico que muestra un título y ejecuta el comando `date`, y luego se prueba en la terminal con `bash ./Ejercicio2_Cobo.sh` para ver la fecha y hora actual del sistema.

VARIABLES – PARAMETROS:



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying a file named `Ejercicio3_Cobo.sh`. The main editor window shows the script content:

```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Saluda al usuario usando el parámetro $1 (nombre).
4
5 if [ -z "$1" ]; then
6     echo "Uso: $0 <nombre>"
7     exit 1
8 fi
9
10 nombre="$1"
11 echo "Hola, $nombre. Bienvenido"
12
```

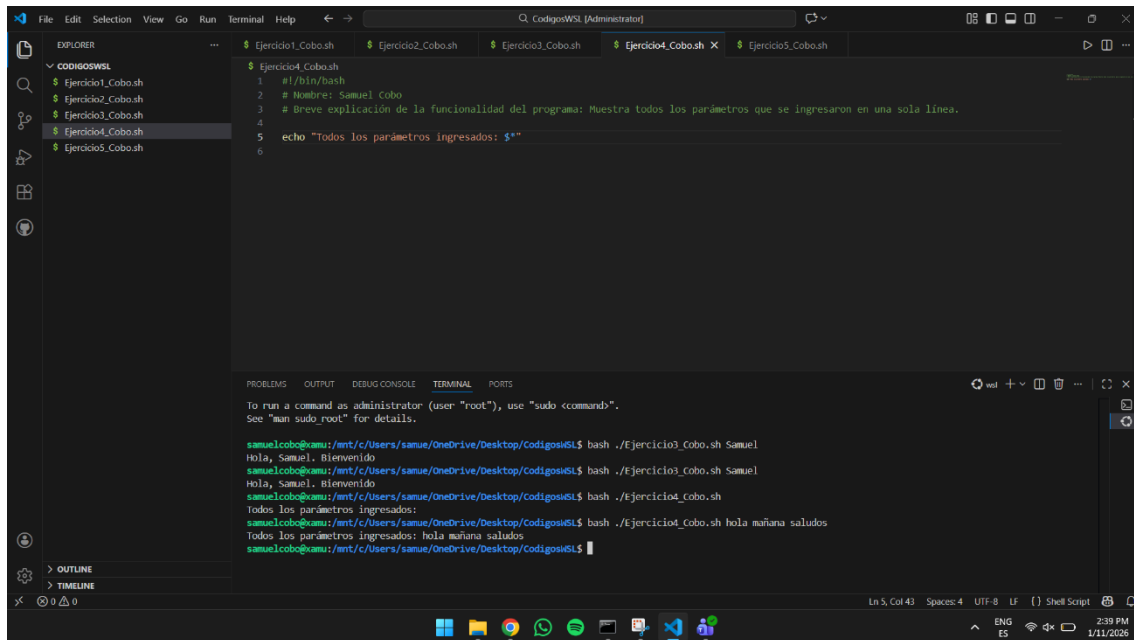
The TERMINAL panel at the bottom shows the execution of the script with a parameter:

```
== PROGRAMA 1 ==
Escribe tu nombre: samuel
Hola, samuel. Bienvenido
samuelcobo@xamui:/mnt/c/Users/samue/OneDrive/Desktop/codigosWSL$ bash ./Ejercicio3_Cobo.sh Samuel
Hola, Samuel. Bienvenido
samuelcobo@xamui:/mnt/c/Users/samue/OneDrive/Desktop/codigosWSL$
```

Paso #4

Descripción: Se edita el script `Ejercicio3_Cobo.sh` para recibir un parámetro (`$1`) como nombre, validar que el usuario lo haya ingresado (si está vacío muestra “Uso: ...” y sale), y luego se ejecuta en la terminal con `bash ./Ejercicio3_Cobo.sh Samuel` para mostrar el saludo usando ese parámetro.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS



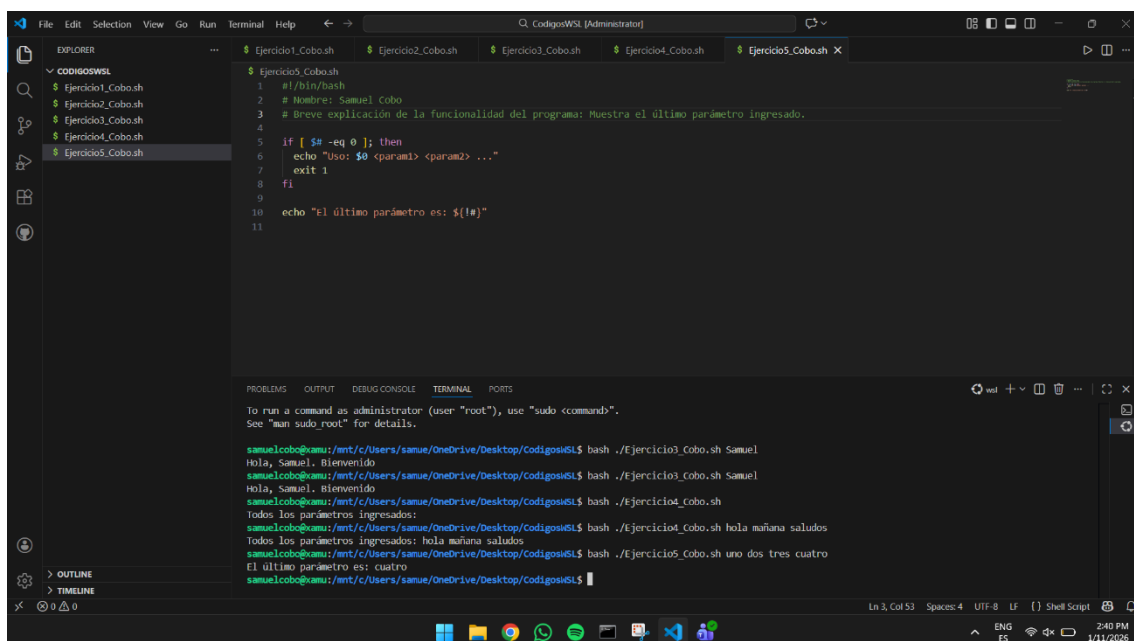
The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying a list of files: Ejercicio1_Cobo.sh, Ejercicio2_Cobo.sh, Ejercicio3_Cobo.sh, Ejercicio4_Cobo.sh, and Ejercicio5_Cobo.sh. The main editor area shows the content of Ejercicio4_Cobo.sh, which is a shell script with the following content:

```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Muestra todos los parámetros que se ingresaron en una sola línea.
4
5 echo "Todos los parámetros ingresados: $*"
6
```

The TERMINAL panel at the bottom shows the execution of the script. The prompt is `samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$`. The first execution is `bash ./Ejercicio4_Cobo.sh Samuel`, which outputs `Hola, Samuel. Bienvenido`. The second execution is `bash ./Ejercicio4_Cobo.sh Samuel`, which outputs `Hola, Samuel. Bienvenido`. The third execution is `bash ./Ejercicio4_Cobo.sh`, which outputs `Todos los parámetros ingresados:`. The fourth execution is `bash ./Ejercicio4_Cobo.sh hola mañana saludos`, which outputs `Todos los parámetros ingresados: hola mañana saludos`. The prompt is `samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$`.

Paso #5

Descripción: Se crea el script Ejercicio4_Cobo.sh para mostrar en una sola línea todos los parámetros que se ingresan al ejecutarlo usando `$*`, y se prueba en la terminal ejecutándolo sin parámetros (sale vacío) y luego con varios (ej: `bash ./Ejercicio4_Cobo.sh hola mañana saludos`) para verificar que los imprime todos juntos.



The screenshot shows the Visual Studio Code interface with the Explorer panel on the left displaying a list of files: Ejercicio1_Cobo.sh, Ejercicio2_Cobo.sh, Ejercicio3_Cobo.sh, Ejercicio4_Cobo.sh, and Ejercicio5_Cobo.sh. The main editor area shows the content of Ejercicio5_Cobo.sh, which is a shell script with the following content:

```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Muestra el último parámetro ingresado.
4
5 if [ $# -eq 0 ]; then
6     echo "Uso: $0 <param1> <param2> ..."
7     exit 1
8 fi
9
10 echo "El último parámetro es: ${!#}"
11
```

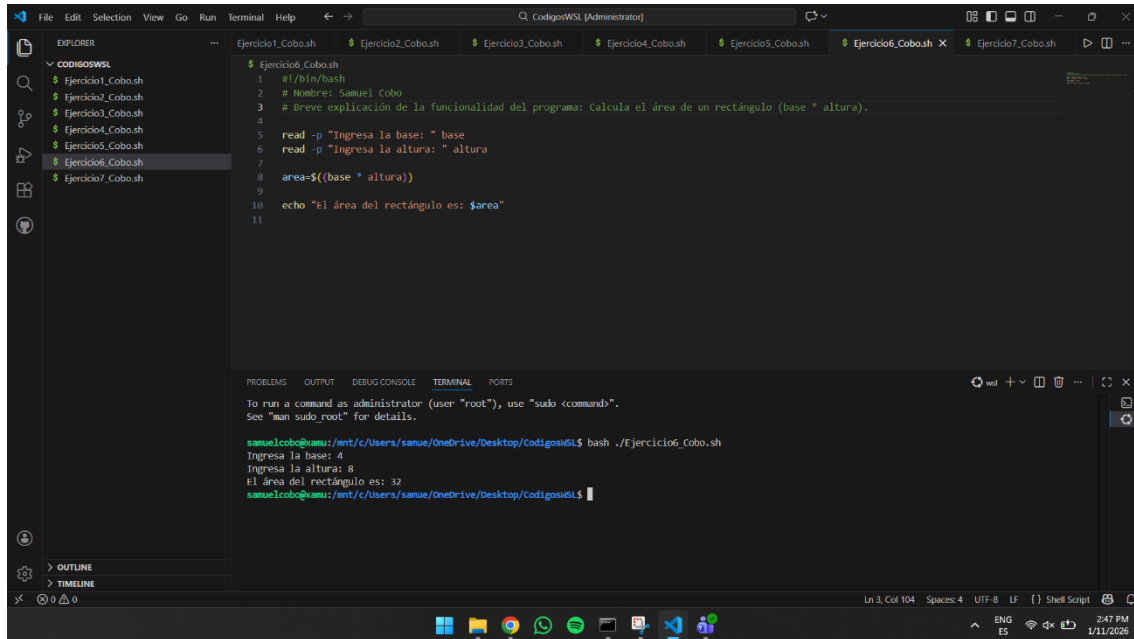
The TERMINAL panel at the bottom shows the execution of the script. The prompt is `samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$`. The first execution is `bash ./Ejercicio5_Cobo.sh Samuel`, which outputs `Hola, Samuel. Bienvenido`. The second execution is `bash ./Ejercicio5_Cobo.sh Samuel`, which outputs `Hola, Samuel. Bienvenido`. The third execution is `bash ./Ejercicio5_Cobo.sh`, which outputs `Todos los parámetros ingresados:`. The fourth execution is `bash ./Ejercicio5_Cobo.sh hola mañana saludos`, which outputs `Todos los parámetros ingresados: hola mañana saludos`. The fifth execution is `bash ./Ejercicio5_Cobo.sh uno dos tres cuatro`, which outputs `El último parámetro es: cuatro`. The prompt is `samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$`.

Paso #6

Descripción: Se desarrolla el script Ejercicio5_Cobo.sh para validar que se ingresen parámetros (usando `$#`) y luego mostrar solo el último parámetro utilizando `${!#}`; finalmente se ejecuta con varios argumentos (ej: `bash ./Ejercicio5_Cobo.sh uno dos tres cuatro`) para comprobar que imprime "cuatro" como último valor.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

EXPRESIONES MATEMÁTICAS:



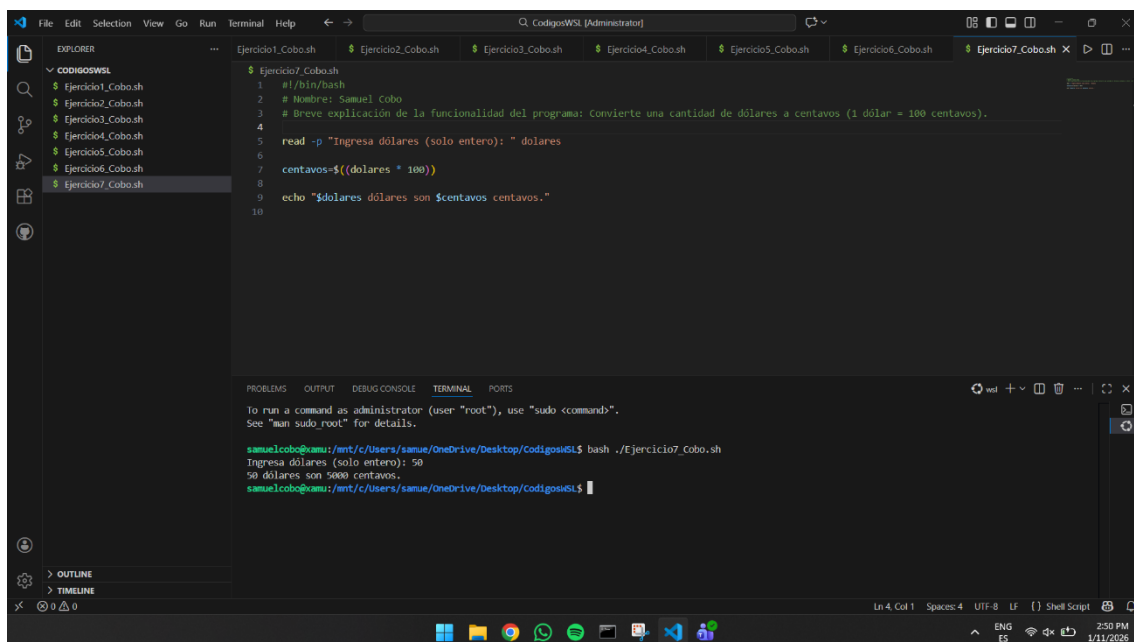
```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Calcula el área de un rectángulo (base * altura).
4
5 read -p "Ingresa la base: " base
6 read -p "Ingresa la altura: " altura
7
8 area=$((base * altura))
9
10 echo "El área del rectángulo es: $area"
11
```

```
to run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

samuelcobo@samu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$ bash ./Ejercicio6_Cobo.sh
Ingresa la base: 4
Ingresa la altura: 8
El área del rectángulo es: 32
samuelcobo@samu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$
```

Paso #7

Descripción: Se crea el script Ejercicio6_Cobo.sh para pedir al usuario la base y la altura de un rectángulo con read, calcular el área usando una expresión aritmética $area=$((base * altura))$, y ejecutarlo con `bash ./Ejercicio6_Cobo.sh` para mostrar el resultado del área en pantalla.



```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Convierte una cantidad de dólares a centavos (1 dólar = 100 centavos).
4
5 read -p "Ingresa dólares (solo entero): " dolares
6
7 centavos=$((dolares * 100))
8
9 echo "$dolares dólares son $centavos centavos."
10
```

```
to run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

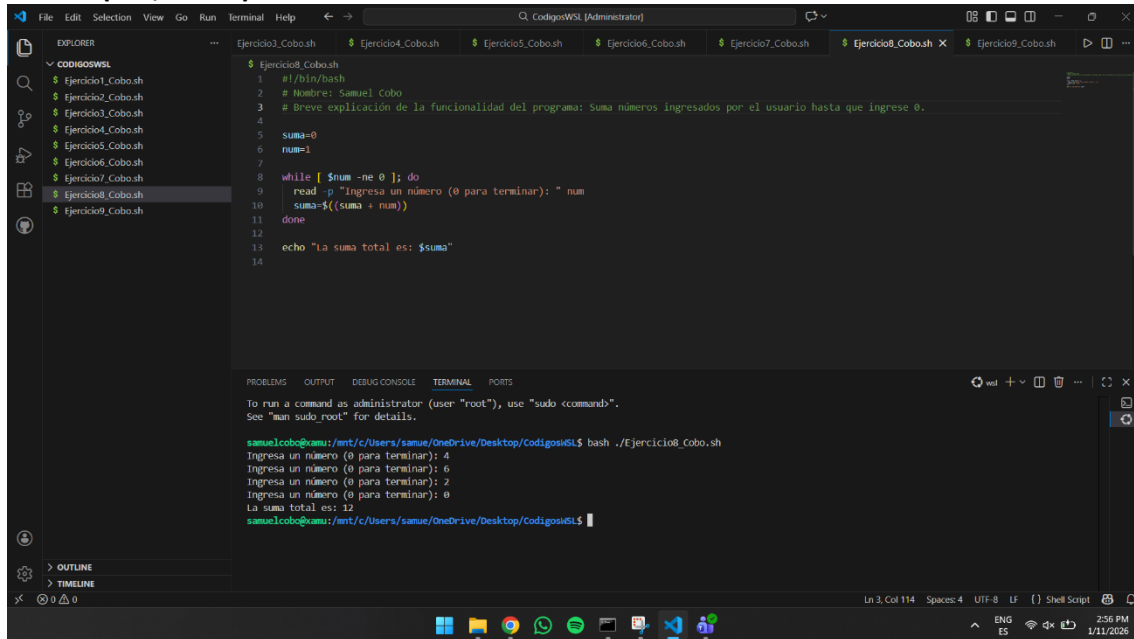
samuelcobo@samu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$ bash ./Ejercicio7_Cobo.sh
Ingresa dólares (solo entero): 50
50 dólares son 5000 centavos.
samuelcobo@samu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$
```

Paso #8

Descripción: Se programa el script Ejercicio7_Cobo.sh para pedir una cantidad de dólares (entero), convertirla a centavos multiplicando por 100 ($centavos=$((dolares * 100))$), y se ejecuta con `bash ./Ejercicio7_Cobo.sh` para mostrar el valor convertido (ejemplo: 50 dólares = 5000 centavos).

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

LAZOS (For, while):



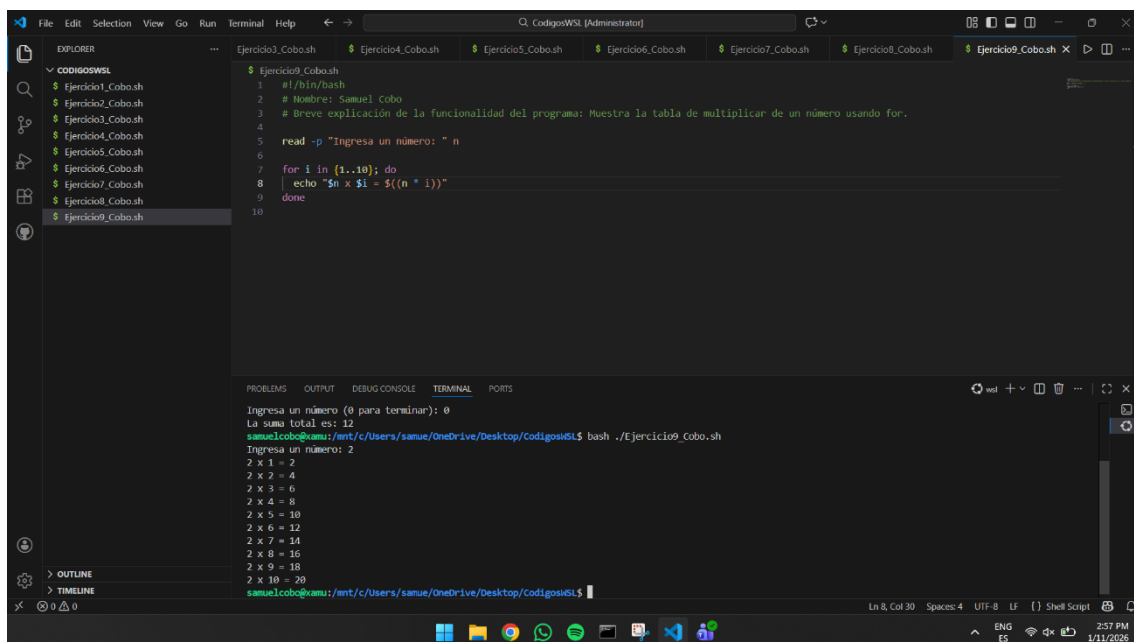
```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Suma números ingresados por el usuario hasta que ingrese 0.
4
5 suma=0
6 num=1
7
8 while [ $num -ne 0 ]; do
9     read -p "Ingresa un número (0 para terminar): " num
10    suma=$((suma + num))
11 done
12
13 echo "La suma total es: $suma"
14
```

```
to run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$ bash ./Ejercicio8_Cobo.sh
Ingresa un número (0 para terminar): 4
Ingresa un número (0 para terminar): 6
Ingresa un número (0 para terminar): 2
Ingresa un número (0 para terminar): 0
La suma total es: 12
samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$
```

Paso #9

Descripción: Se crea el script Ejercicio8_Cobo.sh con un ciclo while que va pidiendo números al usuario y los suma acumuladamente hasta que se ingrese 0; luego se ejecuta con bash ./Ejercicio8_Cobo.sh y al finalizar muestra la suma total de todos los valores ingresados.



```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Muestra la tabla de multiplicar de un número usando for.
4
5 read -p "Ingresa un número: " n
6
7 for i in {1..10}; do
8     echo "$n x $i = ${n * i}"
9 done
10
```

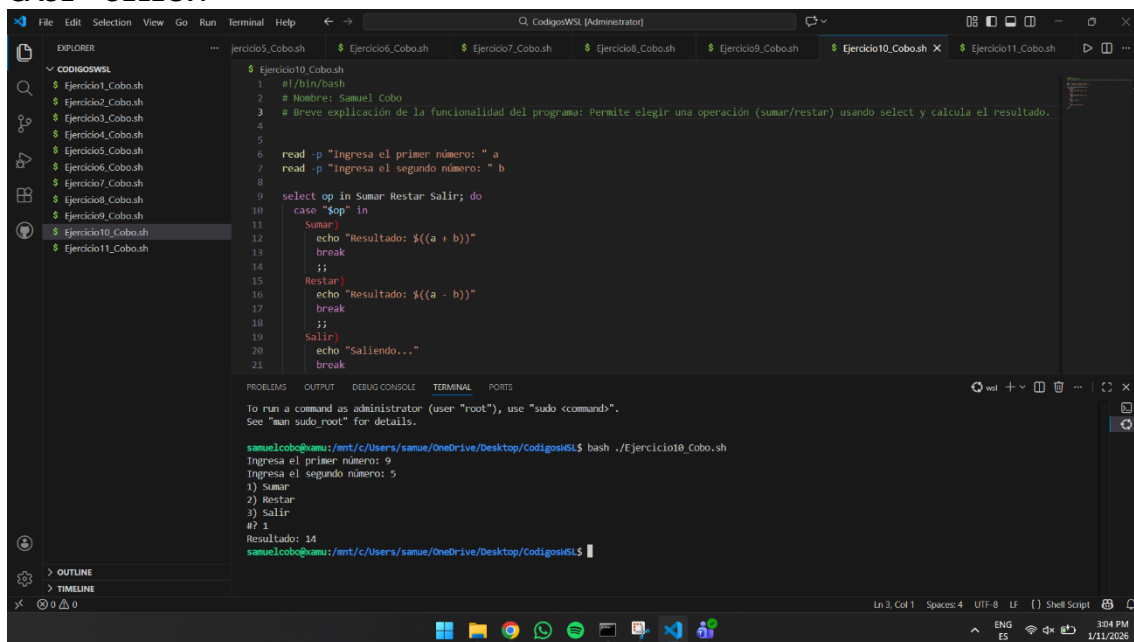
```
Ingresa un número (0 para terminar): 0
La suma total es: 12
samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$ bash ./Ejercicio9_Cobo.sh
Ingresa un número: 2
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
2 x 6 = 12
2 x 7 = 14
2 x 8 = 16
2 x 9 = 18
2 x 10 = 20
samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$
```

Paso #10

Descripción: Se ejecuta el script Ejercicio8_Cobo.sh para comprobar el ciclo while: se ingresan varios números (4, 6, 2...) y al escribir 0 el programa termina y muestra en pantalla la suma total acumulada (en el ejemplo: 12).

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

CASE – SELECT:



```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Permite elegir una operación (sumar/restar) usando select y calcula el resultado.
4
5
6 read -p "Ingresa el primer número: " a
7 read -p "Ingresa el segundo número: " b
8
9 select op in Sumar Restar Salir; do
10 case "$op" in
11     Sumar)
12         echo "Resultado: $((a + b))"
13         break
14         ;;
15     Restar)
16         echo "Resultado: $((a - b))"
17         break
18         ;;
19     Salir)
20         echo "Saliendo..."
21         break
22     esac
23 done
```

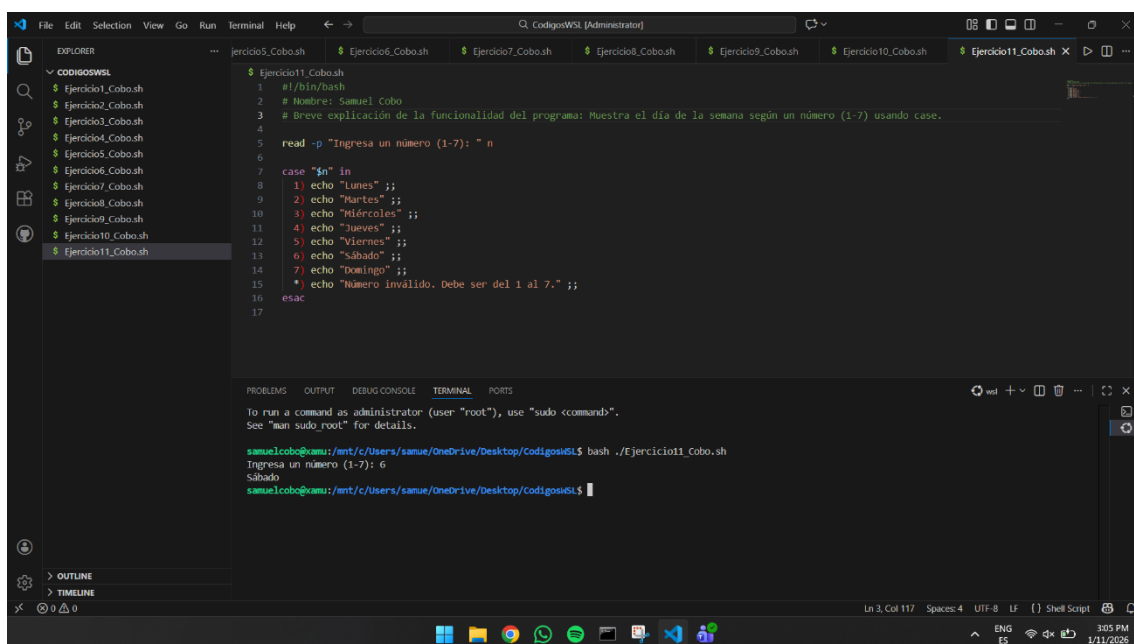
Terminal output:

```
to run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$ bash ./Ejercicio10_Cobo.sh
Ingresa el primer número: 9
Ingresa el segundo número: 5
1) Sumar
2) Restar
3) Salir
#? 1
Resultado: 14
samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$
```

Paso #11

Descripción: Se crea el script Ejercicio9_Cobo.sh para pedir un número y, mediante un ciclo for (for i in {1..10}), mostrar su tabla de multiplicar del 1 al 10; luego se ejecuta con bash ./Ejercicio9_Cobo.sh e ingresando un valor (ej: 2) para imprimir toda la tabla en la terminal.



```
1 #!/bin/bash
2 # Nombre: Samuel Cobo
3 # Breve explicación de la funcionalidad del programa: Muestra el día de la semana según un número (1-7) usando case.
4
5 read -p "Ingresa un número (1-7): " n
6
7 case "$n" in
8     1) echo "Lunes" ;;
9     2) echo "Martes" ;;
10    3) echo "Miércoles" ;;
11    4) echo "Jueves" ;;
12    5) echo "Viernes" ;;
13    6) echo "Sábado" ;;
14    7) echo "Domingo" ;;
15    *) echo "Número inválido. Debe ser del 1 al 7." ;;
16 esac
17
```

Terminal output:

```
to run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

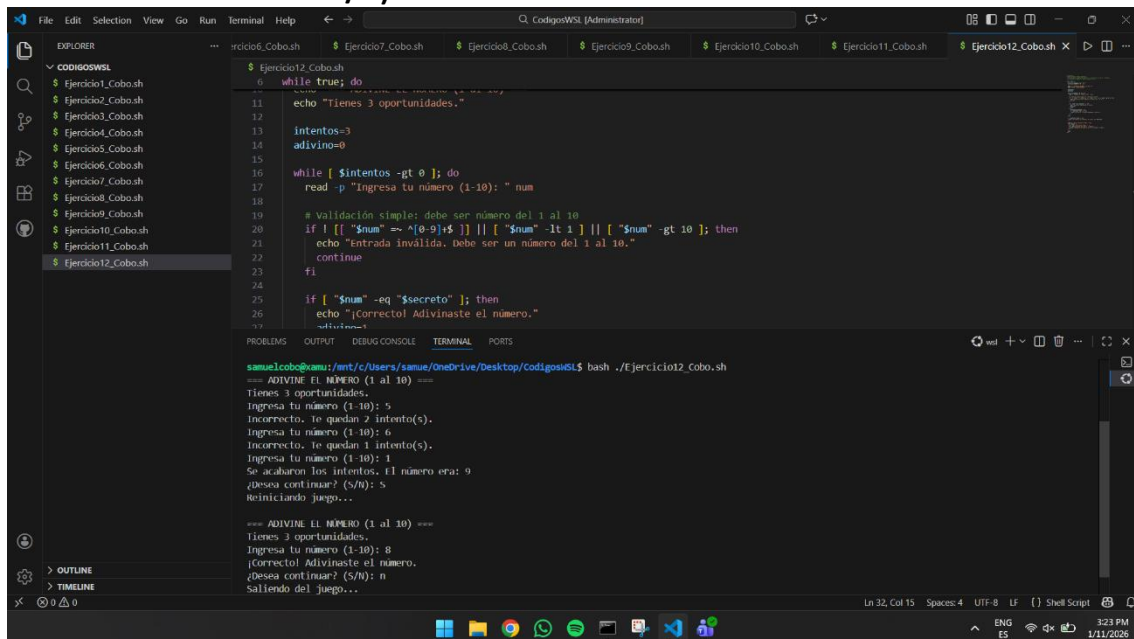
samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$ bash ./Ejercicio11_Cobo.sh
Ingresa un número (1-7): 6
Sábado
samuelcobo@xamu:/mnt/c/Users/samue/OneDrive/Desktop/CodigosWSL$
```

Paso #12

Descripción: Se crea el script Ejercicio11_Cobo.sh usando la estructura case para convertir un número del 1 al 7 en el día de la semana correspondiente; luego se ejecuta con bash ./Ejercicio11_Cobo.sh, se ingresa un número (ej: 6) y el programa muestra el día (ej: "Sábado").

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

JUEGO ADIVINE UN NÚMERO DE 1-10 en tres oportunidades (Según lo indicado en clase ...Desea continuar S/N):



```
 Ejercicio12_Cobo.sh
0 while true; do
11 echo "Tienes 3 oportunidades."
12
13 intentos=3
14 adivino=0
15
16 while [ $intentos -gt 0 ]; do
17 read -p "Ingresa tu número (1-10): " num
18
19 # Validación simple: debe ser número del 1 al 10
20 if ! [[ "$num" =~ ^[0-9]+$ ]] || [ "$num" -lt 1 ] || [ "$num" -gt 10 ]; then
21 echo "Entrada inválida. Debe ser un número del 1 al 10."
22 continue
23 fi
24
25 if [ "$num" -eq "$secreto" ]; then
26 echo "¡Correcto! Adivinaste el número."
27 exit 0
28 fi
29
30 intentos=$((intentos-1))
31
32 done
33
34 echo "Se acabaron los intentos. El número era: $secreto"
35
36 # Desea continuar? (S/N)
37 read -p "¿Desea continuar? (S/N): " continuar
38
39 if [ "$continuar" = "S" ]; then
40 echo "Reiniciando juego..."
41
42 # Reiniciar juego
43
44 else
45 echo "Saliendo del juego..."
46 exit 0
47 fi
48
49 done
```

```
 samuelcobo@samuel:/mnt/c/Users/samuel/OneDrive/Desktop/CodigoWSL$ bash ./Ejercicio12_Cobo.sh
=== ADIVINE EL NÚMERO (1 al 10) ===
Tienes 3 oportunidades.
Ingresa tu número (1-10): 5
Incorrecto. Te quedan 2 intento(s).
Ingresa tu número (1-10): 6
Incorrecto. Te quedan 1 intento(s).
Ingresa tu número (1-10): 1
Se acabaron los intentos. El número era: 9
¿Desea continuar? (S/N): n
Reiniciando juego...

=== ADIVINE EL NÚMERO (1 al 10) ===
Tienes 3 oportunidades.
Ingresa tu número (1-10): 8
¡Correcto! Adivinaste el número.
¿Desea continuar? (S/N): n
Saliendo del juego...
```

Paso #13

Descripción: Se ejecuta el script Ejercicio12_Cobo.sh (juego "Adivine el número 1–10") donde el usuario tiene 3 intentos para adivinar; el programa valida la entrada, indica si fue correcto/incorrecto, muestra cuántos intentos quedan y al terminar pregunta si desea continuar (S/N) para reiniciar el juego o salir.

V. ANALISIS DE RESULTADOS

Los resultados obtenidos demuestran que el entorno WSL permitió ejecutar correctamente todos los scripts en Bash, validando el flujo completo de trabajo: creación de archivos, edición en VS Code y pruebas desde la terminal. En los ejercicios básicos se verificó la salida esperada (saludo, fecha/hora) y en los de parámetros se comprobó que las validaciones funcionaron adecuadamente, mostrando mensajes de uso cuando faltaban argumentos y procesando correctamente listas de parámetros y el último valor ingresado.

En los ejercicios con expresiones matemáticas y estructuras de control, los cálculos y repeticiones se ejecutaron de forma consistente con los datos ingresados. El while permitió acumular valores hasta la condición de salida (0), el for generó la tabla de multiplicar completa y el case devolvió el día correspondiente según el número ingresado, incluyendo manejo de entradas inválidas. Finalmente, el juego de adivinanza confirmó el uso integrado de validación, conteo de intentos, mensajes de retroalimentación y opción de reinicio (S/N), cumpliendo con el comportamiento solicitado.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

VI. CONCLUSIONES

En conclusión, se logró desarrollar y ejecutar correctamente los scripts en Bash utilizando WSL (Ubuntu), aplicando de forma práctica los conceptos fundamentales de programación en shell. A través de los ejercicios se consolidó el manejo de entrada y salida, variables, parámetros y operaciones aritméticas, además de validar datos para evitar errores durante la ejecución.

Asimismo, la implementación de estructuras de control como for, while y case permitió resolver problemas más completos y estructurados, culminando en un juego de adivinanza con intentos limitados y opción de continuar. En general, el trabajo evidenció un aprendizaje progresivo y funcional del uso de Bash para automatizar tareas y construir programas simples pero bien organizados.

VII. RECOMENDACIONES

- **Dar permisos de ejecución** a los scripts para ejecutarlos directamente y mantener una forma estándar de ejecución.
- **Mantener una estructura y documentación uniforme** en todos los archivos: encabezado con nombre del desarrollador, objetivo del programa y comentarios breves por secciones.
- **Agregar validaciones más completas** (por ejemplo: asegurar que las entradas sean numéricas cuando se requiera) usando expresiones regulares o comprobaciones con case/[[]].
- **Controlar errores y salidas** con códigos para que el script indique claramente si terminó bien o falló por mal uso.
- **Organizar los scripts por carpetas** (básicos, parámetros, matemáticas, lazos, case, juego) para que el proyecto sea más fácil de revisar y presentar.

VIII. ANEXOS

SIMULACIONES:

Examen 1:

The screenshot shows a web browser window displaying a score report on the Gmetrix.net platform. The user is identified as Samuel Francis. The interface includes a sidebar with navigation options: Inicio, Transcripción, Activar código, Cuenta, Ayuda, Cerrar sesión, Idioma, Descargar SMS, and Light Mode. The main content area is titled 'Test Attempts' and lists several attempts with their dates and times. A 'Summary' tab is selected, showing a 'My Score' of 975, which is 'Aprobado' (Approved). A gauge chart visualizes the score, with a green needle pointing to 975 on a scale from 0 to 1000. A button 'Revisar preguntas omitidas (5 Reviews Available)' is present. The 'Test Details' section shows the category as 'Cisco- CCST', the product as 'Cisco Certified Support Technician: IT Support', the mode as 'Pruebas', and the passing score as 700. A 'Print' button is also visible.

Test Attempts

- ✓ Pruebas | 01/04/2026 07:08 PM
- ✓ Entrenamiento | 01/04/2026 06:57 PM
- ✓ Entrenamiento | 01/04/2026 06:54 PM
- ✓ Entrenamiento | 01/04/2026 06:49 PM
- ✓ Entrenamiento | 01/04/2026 06:41 PM
- ✓ Entrenamiento | 01/04/2026 05:52 PM
- ✓ Pruebas | 12/17/2025 07:52 AM
- ✓ Entrenamiento | 12/15/2025 11:46 AM
- ✗ Pruebas | 12/10/2025 01:21 PM

Summary | Objectives | Preguntas

Revisar preguntas omitidas (5 Reviews Available) | Imprimir

My Score ▲ 200.00 From Last Attempt

Aprobado
975

Test Details

Categoría Cisco- CCST	Producto Cisco Certified Support Technician: IT Support
Modo Pruebas	Passing Score 700

CMETRIX
Estudiante

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Examen 2:

SAMUEL FRANCIS...

Inicio

Transcripción

Activar código

Cuenta

Ayuda

Cerrar sesión

Idioma

Descargar SMS

Light Mode

CMETRIX

Estudiante

< Cisco Certified Support Technician: IT Support Practice Exam 2

Test Attempts

✓ Pruebas | 01/11/2026 01:51 PM

✓ Pruebas | 01/05/2026 09:36 AM

✓ Pruebas | 01/04/2026 07:54 PM

✓ Pruebas | 01/04/2026 07:49 PM

✓ Pruebas | 01/04/2026 07:42 PM

✓ Entrenamiento | 01/04/2026 07:27 PM

✓ Entrenamiento | 12/17/2025 12:17 PM

✓ Entrenamiento | 12/17/2025 07:35 AM

✓ Entrenamiento | 12/15/2025 12:38 PM

Summary

Objectives

Preguntas

Revisar preguntas omitidas (5 Reviews Available)

Imprimir

My Score

▼ -25.00

From Last Attempt

Aprobado

975

Test Details

Categoría

Cisco- CCST

Producto

Cisco Certified Support Technician: IT

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

CERTIFICADOS:



This certificate is awarded to

samuel cobo

for successfully completing

IT Customer Support Basics

offered by Networking Academy
through the Cisco Networking Academy program.

Lynn Bloomer

Lynn Bloomer
Director
Cisco Networking Academy

11 Jan 2026
Completion Date



This certificate is awarded to

samuel cobo

for successfully completing

Operating Systems Support

offered by Networking Academy
through the Cisco Networking Academy program.

Lynn Bloomer

Lynn Bloomer
Director
Cisco Networking Academy

05 Jan 2026
Completion Date

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS



This certificate is awarded to

samuel cobo

for successfully completing

Security and Connectivity Support

offered by Networking Academy
through the Cisco Networking Academy program.

Lynn Bloomer

Lynn Bloomer
Director
Cisco Networking Academy

05 Jan 2026
Completion Date



This certificate is awarded to

samuel cobo

for successfully completing

Hardware and Upgrade Support

offered by Networking Academy
through the Cisco Networking Academy program.

Lynn Bloomer

Lynn Bloomer
Director
Cisco Networking Academy

11 Jan 2026
Completion Date

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

ACTIVIDAD 1:



ACTIVIDAD 2:

Lecturas base:

1. *AIOS: LLM Agent Operating System* (noviembre 2024).
2. *AI for Next Generation Computing: Emerging Trends and Future Directions* (febrero 2022).

1. Objetivo del debate

Discutir y priorizar los principales desafíos y oportunidades que surgen al integrar Inteligencia Artificial a nivel de sistema operativo, considerando implicaciones técnicas, de seguridad y de impacto social.

2. Roles y perspectivas asignadas

Julio — Eficiencia técnica y escalabilidad

Jeremy — Capacidades de la IA y futuro de los SO

Samuel — Seguridad, ética e impacto social

3. Desarrollo resumido del panel

3.1 Aperturas

Julio:

Argumenta que sin un mecanismo tipo kernel, múltiples agentes pueden competir de forma desordenada por recursos, generando demoras y colisiones. Propone la necesidad de scheduling y administración del contexto y memoria para mejorar rendimiento.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

Jeremy:

Plantea que los LLM y agentes están pasando de ser simples aplicaciones a convertirse en componentes cercanos al sistema operativo, lo que podría transformar la interacción humano-computador y crear nuevas “aplicaciones-agente”.

Samuel:

Advierte que integrar IA a nivel de SO aumenta la superficie de ataque y los riesgos: seguridad entre agentes, privacidad y control de acciones. Indica que la gobernanza debe ser un requisito de diseño, no un añadido final.

3.2 Ronda 1 — Escalabilidad y rendimiento

Pregunta guía: ¿Por qué se necesita un “SO para agentes” y no solo frameworks?

- **Julio:** sostiene que un sistema tipo AIOS permite coordinar y repartir recursos para que ningún agente monopolice el LLM, manteniendo eficiencia al escalar.
- **Samuel:** responde que la centralización también aumenta impacto si hay fallos o vulnerabilidades.
- **Jeremy:** modera indicando que la escalabilidad debe equilibrarse con control y transparencia.

3.3 Ronda 2 — Seguridad, privacidad y control

Pregunta guía: ¿Cómo evitar que un agente acceda a memoria de otro o ejecute acciones peligrosas?

- **Samuel:** prioriza controles: aislamiento entre agentes, permisos, auditoría y trazabilidad.
- **Julio:** reconoce el trade off: cooperación entre agentes requiere memoria compartida, pero con control estricto.
- **Jeremy:** concluye que la seguridad debe diseñarse al nivel de kernel/servicios para ser consistente en todo el ecosistema.

3.4 Ronda 3 — Futuro e impacto social

Pregunta guía: ¿Qué pasa con transparencia, confianza y regulación si el sistema se vuelve “autónomo”?

- **Jeremy:** destaca el objetivo de sistemas autogestionados para infraestructuras complejas.
- **Samuel:** enfatiza que sin estándares y regulación, especialmente en entornos edge/cloud, aumentan riesgos y dependencia de proveedores.
- **Julio:** añade que la autonomía debe medirse y limitarse por políticas de recursos y seguridad verificables.

FACULTAD DE INGENIERÍA Y CIENCIAS APLICADAS

4. Priorización final del comité

4.1 Desafíos priorizados:

Seguridad y aislamiento entre agentes.

Scheduling justo y control de concurrencia.

1. Escalabilidad predecible con múltiples agentes concurrentes.
2. Eficiencia en manejo de contexto/memoria.
3. Gobernanza: estándares, auditoría y transparencia para adopción responsable.

4.2 Oportunidades priorizadas

1. Mejora del rendimiento y organización al sistematizar agentes como servicios del SO.
2. Evolución hacia sistemas autogestionados en infraestructuras modernas.
3. Creación de ecosistemas de apps agente con servicios reutilizables.
4. Mejor coordinación en cloud/Edge/serverless con IA para gestión y optimización.
5. Trazabilidad y protección.

5. Conclusión

El comité concluye que la integración de IA a nivel de sistema operativo ofrece beneficios claros de rendimiento y automatización, pero exige priorizar seguridad, aislamiento, gobernanza y control de recursos. Se recomienda que cualquier SO orientado a agentes incorpore desde su diseño mecanismos de permisos, auditoría y scheduling, con transparencia y estándares para uso responsable.

IX. BIBLIOGRAFÍA COMPLEMENTARIA

- Greenberg, M., Kallas, K., & Vasilakis, N. (2021). *Unix shell programming: The next 50 years*. In *Workshop on Hot Topics in Operating Systems (HotOS '21)* (pp. 104–111). Association for Computing Machinery.
<https://doi.org/10.1145/3458336.3465294>
- Mei, K., Zhu, X., Xu, W., Hua, W., Jin, M., Li, Z., Xu, S., Ye, R., Ge, Y., & Zhang, Y. (2024). *AIOS: LLM agent operating system* (arXiv:2403.16971). arXiv.
- Gill, S. S., Xu, M., Ottaviani, C., Patros, P., Bahsoon, R., Shaghaghi, A., Golec, M., Stankovski, V., Wu, H., Abraham, A., Singh, M., Mehta, H., Ghosh, S. K., Baker, T., Puthal, D., Varghese, B., Jamalipour, A., & Buyya, R. (2022). *AI for next generation computing: Emerging trends and future directions* (Preprint accepted for publication in *Elsevier IoT Journal*).